# コンパイラ

学籍番号：6324029
氏名：北澤泰樹

提出日：2026 年 8 月 15 日

# 1 課題１

## 1.1 修正箇所の説明

字句解析器を定義する lexer.mll を修正した. 通常の字句解析規則（lexer）に, 文字列 "//" を検知した際の規則を追加し, 新たに定義した comment 規則へ処理を遷移させるようにした. 追加した comment 規則では, 改行文字（'\n'）を検知するまでは入力された任意の文字（_）を読み捨てて自身の規則を再帰的に呼び出す. そして, 改行文字を検知した時点で通常の lexer 規則へ復帰させるアクションを記述することで, 行末までの文字列をコメントとして無視する処理を実現した.

## 1.2 ソースコード

```
(* File lexer.mll *)
{
 open Parser
 exception No_such_symbol
}

let digit = ['0'-'9']
let id = ['a'-'z' 'A'-'Z' '_'] ['a'-'z' 'A'-'Z' '0'-'9']*

rule lexer = parse
| digit+ as num  { NUM (int_of_string num) }
| "if"                    { IF }
| "else"                  { ELSE }
| "while"                 { WHILE }
| "scan"                  { SCAN }
| "sprint"                { SPRINT }
| "iprint"                { IPRINT }
| "int"                   { INT }
| "return"                { RETURN }
| "type"                  { TYPE }
| "void"                  { VOID }
| id as text              { ID text }
| '\"'[^'\"']*'\"' as str { STR str }
| '='                     { ASSIGN }
| "=="                    { EQ }
| "!="                    { NEQ }
| '>'                     { GT }
| '<'                     { LT }
```

```
| ">="                    { GE }
| "<="                    { LE }
| '+'                     { PLUS }
| '-'                     { MINUS }
| '*'                     { TIMES }
| '/'                     { DIV }
| '{'                     { LB  }
| '}'                     { RB  }
| '['                     { LS }
| ']'                     { RS }
| '('                     { LP  }
| ')'                     { RP  }
| ','                     { COMMA }
| ';'                     { SEMI }
(*課題１で追加*)
|"//"                      {comment lexbuf}
| [' ' '\t']          { lexer lexbuf }(* eat up whitespace *)
|['\n']                 {Lexing.new_line lexbuf;lexer lexbuf}
| eof                     { raise End_of_file }
| _                       { raise No_such_symbol }

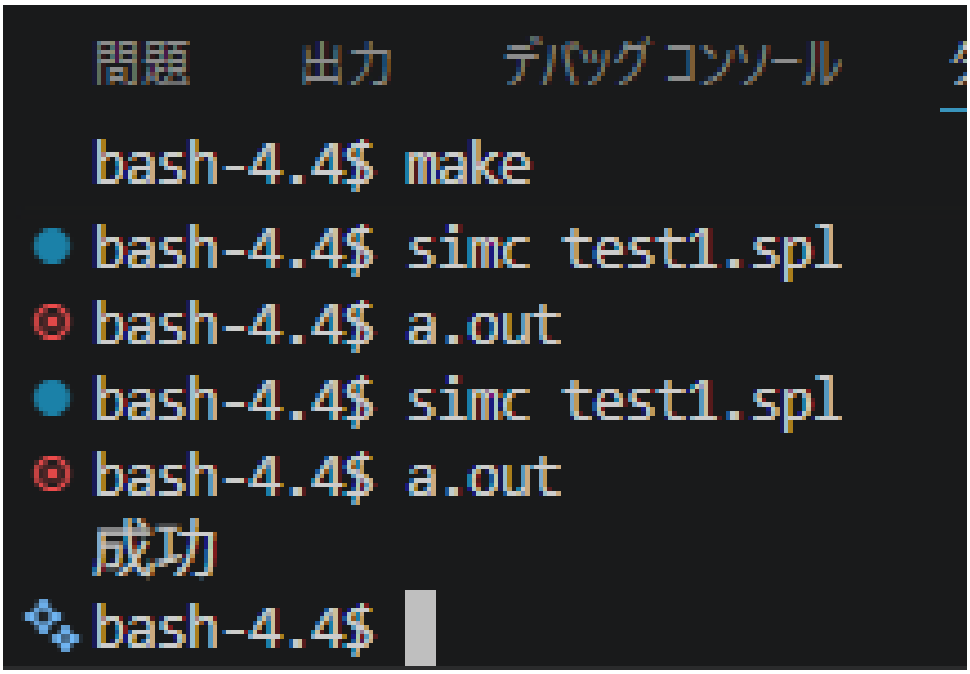
(*課題 1comment 用のルール*)
and comment = parse
|'\n'                     {Lexing.new_line lexbuf; lexer lexbuf}
|eof                      {raise End_of_file}
|_                        {comment lexbuf}
```

## 1.3 アセンブリコード

```
  IO:
.string "%lld"
.text
.globl main
main:
pushq %rbp
movq %rsp, %rbp
subq $16, %rsp
.data
L1: .string "成功\n"
```

```
.text
leaq L1(%rip), %rdi
movq $0, %rax
callq printf
leaveq
retq
```

### 1.4 実行結果



# 2 課題２

## 2.1 修正箇所の説明

- **字句解析器（lexer.mll）の修正**
  従来は改行文字（'\n'）を単に読み飛ばす処理のみを行っていたが, 改行文字を検知した際に Lexing.new_line lexbuf 関数を呼び出すアクションを新たに追加した. これにより, 字句解析の進行に伴って現在の行番号が内部で正しく追跡・記録されるようになった.
- **メイン処理（sim.ml）の修正**
  構文解析器の呼び出し処理（Parser.prog）において発生する例外 Parsing.Parse_error を捕捉するため, 当該処理全体を try ... with 文で囲む形に変更した. エラーを検知した際には, 字句バッファ（lexbuf）の現在の状態から lexbuf.Lexing.lex_curr_p.Lexing.pos_lnum を用いて行番号を取得するとともに,Lexing.lexeme lexbuf を用いて直後の字句（トークン）の文字列を取得し,Printf.printf によって画面に出力するようにした. また, 例外処理ブロックから変数 lexbuf を参照できるようにするため, try ... with 文全体を main 関数のスコープ内へ移動する構造変更を行った.

## 2.2 ソースコード

### 2.2.1 lexer.mll の修正箇所

```
(* 修正箇所：lexer.mll の末尾付近 *)
| [' ' '\t']              { lexer lexbuf }
| '\n'                    { Lexing.new_line lexbuf; lexer lexbuf } (* 追加: 改行時に行番号をカウント *)
```

### 2.2.2 sim.ml の修正箇所

```
(* 修正箇所：sim.ml の main 関数内 *)
  try
    (*（中略: コード生成やアセンブラ呼び出しなどの既存処理）*)
    let _ = Unix.system "gcc tmp.s" in ()

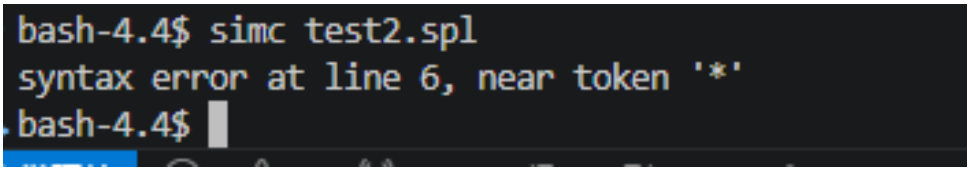
  with (* 課題２：エラーが起きた行番号と直後の字句を表示 *)
  | Parsing.Parse_error ->
      let pos = lexbuf.Lexing.lex_curr_p in
      let line = pos.Lexing.pos_lnum in
      let tok = Lexing.lexeme lexbuf in
      Printf.printf "syntax error at line %d, near token '%s'\n" line tok
  | Table.No_such_symbol x -> print_string ("no such symbol: \""^x^"\"\n")
```

```
  | Semant.TypeErr s -> print_string (s^"\n")
  | Semant.Err s -> print_string (s^"\n")
  | Table.SymErr s -> print_string (s^"\n")
```

#### 2.2.3 アセンブリコード

エラー処理についてなので課題１と同じものが出力された状態のため、省略する。

## 2.3 実行結果



# 3　課題３

## 3.1　修正箇所の説明

構文解析中に文法エラー（構文エラー）が発生した際, コンパイラが直ちにパース処理を中断してパニックに陥るのを防ぐため, エラー回復（Error Recovery）機構を導入した. 具体的には, 構文解析器（parser.mly）の文（stmt）の生成規則の末尾に,ocamlyacc に組み込まれている特殊なトークン error を用いた規則| error SEMI { NilStmt } を追加した. これにより, ある文の途中で予期せぬトークンが現れて構文エラーが発生した場合でも, エラー位置から次のセミコロン（SEMI）までの入力を読み飛ばし, その文を無効な空文（NilStmt）として扱うことで内部状態をリセットし, 次の文から正常にパース処理を再開できるようになった.

## 3.2　ソースコード

### 3.2.1　parser.mly の修正箇所

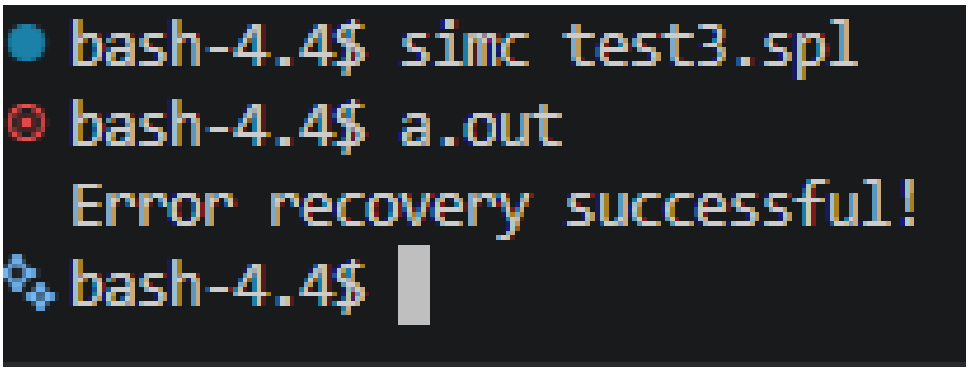
```
(* 修正箇所：parser.mly の stmt（文）のルール部分の末尾 *)

stmt : ID ASSIGN expr SEMI    { Assign (Var $1, $3) }
     | ID PLUSEQ expr SEMI    { Assign (Var $1, CallFunc ("+", [VarExp (Var $1); $3])) }
     /* ...（中略）... */
     | RETURN expr SEMI       { CallProc ("return", [$2]) }
     | block                  { $1 }
     | SEMI                   { NilStmt }
     | error SEMI             { NilStmt }  /* 追加: 構文エラー時にセミコロンまで読み飛ばして回復するルール */
     ;
```

### 3.2.2　アセンブリコード

課題２と同様に省略する。

### 3.3 実行結果


bash-4.4$ simc test3.spl
bash-4.4$ a.out
Error recovery successful!
bash-4.4$

# 4 課題 4

## 4.1 修正箇所の説明

剰余演算子 % を追加するにあたり, 抽象構文木（AST）やコード生成部を新たに拡張するのではなく, 構文解析の段階で A % B を A - (A / B) * B という既存の四則演算の組み合わせに展開するアプローチ（構文糖衣）を採用した. この実装のため,lexer.mll と parser.mly の 2 ファイルを修正した.

- **字句解析器（lexer.mll）の修正**
  演算子の解析規則に % を検知する規則を追加し, 新たに定義した MOD トークンを返すように修正した.
- **構文解析器（parser.mly）の修正**
  1. %token の定義に MOD を追加し, 新しいトークンとして宣言した.
  2. 演算子の優先順位と結合規則の定義において, 掛け算（TIMES）および割り算（DIV）と同じ優先度の行に %left MOD として追加した.
  3. 数式の生成規則(expr:)に | expr MOD expr の規則を追加した. このアクション内で, 左辺($1)と右辺($3)を用いて CallFunc ("-", [$1; CallFunc ("*", [CallFunc ("/", [$1; $3]); $3])]) を返すように記述した. これにより,A % B という式が, 既存の構文木構造を用いた A - (A / B) * B に自動的に変換される.

この手法により, 意味解析（semant.ml）やコード生成（emitter.ml）のソースコードを一切変更することなく, 剰余演算を正しく処理させることが可能となった.

## 4.2 ソースコード

### 4.2.1 lexer.mll の修正箇所

```
(* 修正箇所：lexer.mll の演算子定義部分 *)
| '*'                     { TIMES }
| '/'                     { DIV }
| '%'                     { MOD }
```

### 4.2.2 parser.mly の修正箇所

```
(* 修正箇所^^e2^^91^^a0：parser.mly の %token 宣言部分 *)
%token PLUS MINUS TIMES DIV MOD LB RB LS RS LP RP ASSIGN SEMI COMMA TYPE VOID

(* 修正箇所^^e2^^91^^a1：parser.mly の優先順位定義部分 *)
%left PLUS MINUS        /* lowest precedence */
%left TIMES DIV MOD     /* medium precedence */
%nonassoc UMINUS        /* highest precedence */

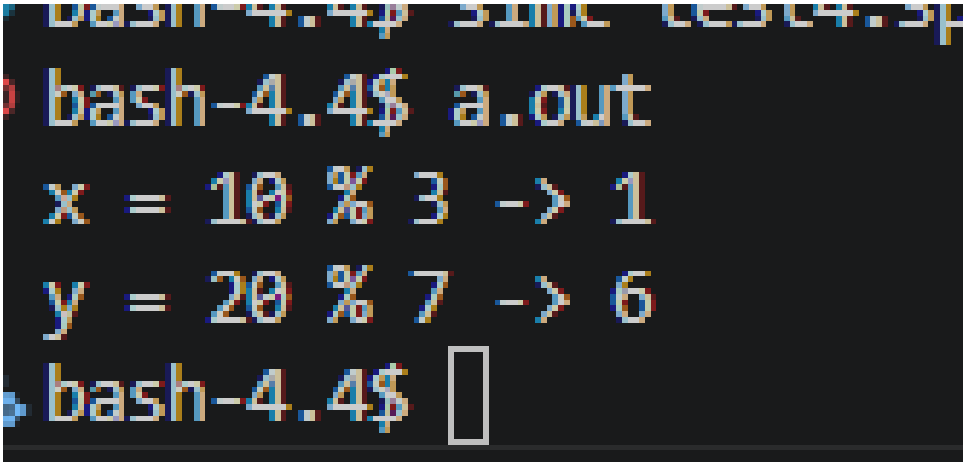
(* 修正箇所^^e2^^91^^a2：parser.mly の expr（式）のルール部分 *)
```

```
    | expr PLUS expr { CallFunc ("+", [$1; $3]) }
    | expr MINUS expr { CallFunc ("-", [$1; $3]) }
    | expr TIMES expr { CallFunc ("*", [$1; $3]) }
    | expr DIV expr { CallFunc ("/", [$1; $3]) }
    | expr MOD expr { CallFunc ("-", [$1; CallFunc ("*", [CallFunc ("/", [$1; $3]); $3])]) }
```

#### 4.2.3 アセンブリコード

```
 pushq $10
pushq $10
pushq $3
popq %rbx
popq %rax
cqto
idivq %rbx
pushq %rax
pushq $3
popq %rax
imulq (%rsp), %rax
movq %rax, (%rsp)
popq %rax
subq %rax, (%rsp)
movq %rbp, %rax
```

### 4.3 実行結果



# 5 課題5

## 5.1 修正箇所の説明

## 5.2 ソースコード

### 5.2.1 ast.ml の修正箇所

```
(* 修正箇所：ast.ml の dec の定義部分 *)
and dec = FuncDec of id * ((typ*id) list) * typ * stmt
        | TypeDec of id * typ
        | VarDec of typ * id
        | VarDecInit of typ * id * exp  (* 追加: 初期値付き変数宣言 *)
```

### 5.2.2 parser.mly の修正箇所

```
(* 修正箇所：parser.mly の dec のルール部分 *)
dec   : ty ids SEMI   { List.map (fun x -> VarDec ($1,x)) $2 }
      | TYPE ID ASSIGN ty SEMI { [TypeDec ($2,$4)] }
      | ty ID LP fargs_opt RP block  { [FuncDec($2, $4, $1, $6)] }
      | VOID ID LP fargs_opt RP block  { [FuncDec($2, $4, VoidTyp, $6)] }
      | ty ID ASSIGN expr SEMI { [VarDecInit ($1, $2, $4)] } /*  追加: 変数宣言と初期
化 */
      ;
```

### 5.2.3 semant.ml の修正箇所

```
(* 修正箇所^^e2^^91^^a0：semant.ml の check_redecl 関数の中 *)
       | VarDec (_,s)::rest -> if List.mem s vl then raise (SymErr s)
                               else check_redecl rest tl (s::vl)
       | VarDecInit (_,s,_)::rest -> if List.mem s vl then raise (SymErr s) (*  追
加 *)
                                     else check_redecl rest tl (s::vl)
       | TypeDec (s,_)::rest -> if List.mem s tl then raise (SymErr s)


(* 修正箇所^^e2^^91^^a1：semant.ml の type_dec 関数の中 *)
    (* 変数宣言の処理 *)
    | VarDec (t,s) -> (tenv,
              update s (VarEntry {ty= create_ty t tenv; offset=addr-8; level=nest}) env, addr-8)
    (* 追加: 初期値付き変数宣言の処理 *)
    | VarDecInit (t,s,e) ->
```

```
        let declared_ty = create_ty t tenv in
        let expr_ty = type_exp e env in
        if (actual_ty declared_ty) != expr_ty then raise (TypeErr "type error in initialization");
        (tenv,
         update s (VarEntry {ty= declared_ty; offset=addr-8; level=nest}) env, addr-8)
```

### 5.2.4 emitter.ml の修正箇所

```
(* 修正箇所^^e2^^91^^a0：emitter.ml の trans_dec 関数内 *)
   (* 変数宣言の処理 *)
 | VarDec (t,s) -> ()
 | VarDecInit (t,s,e) -> () (*  追加: 領域確保のみでコードは出力しない *)
   (* 型宣言の処理 *)
 | TypeDec (s,t) ->

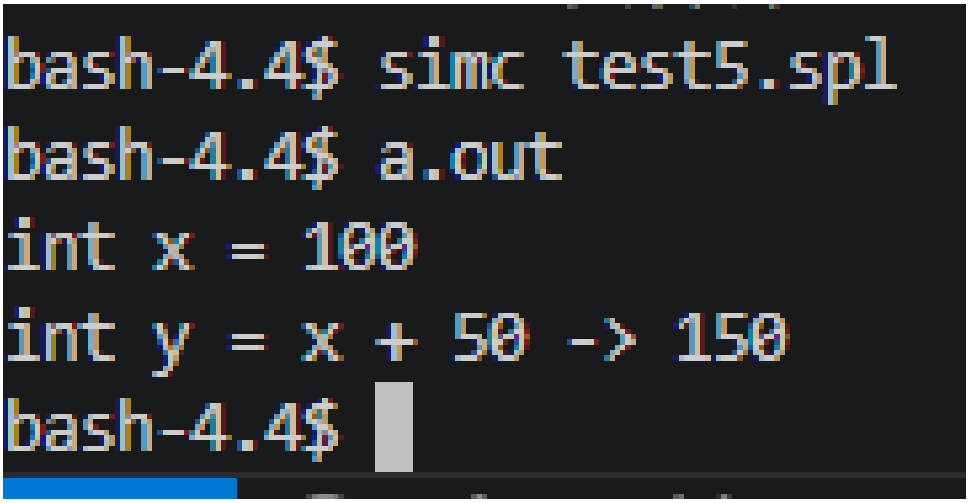

(* 修正箇所^^e2^^91^^a1：emitter.ml の trans_stmt 関数内、Block の処理 *)
                  (* ブロックのコード：文を表すブロックは，関数定義を無視する．*)
                  | Block (dl, sl) ->
                       (* ブロック内宣言の処理 *)
                       let (tenv',env',addr') = type_decs dl nest tenv env in
                             List.iter (fun d -> trans_dec d nest tenv' env') dl;
                             (* フレームの拡張 *)
                             let ex_frame = sprintf "\tsubq $%d, %%rsp\n" ((-addr'+16)/16*16) in
                                  (*  追加: 初期値付き変数の代入コードを自動生成する *)
                                  let init_code = List.fold_left (fun acc d ->
                                      match d with
                                      | VarDecInit (_, s, e) -> acc ^ trans_stmt (Assign (Var s, e))
                                      | _ -> acc
                                  ) "" dl in
                                  (* 本体（文列）のコード生成 *)
                                  let code = List.fold_left
                                       (fun code ast -> (code ^ trans_stmt ast nest tenv' env')) ""
                                  (*  修正: ex_frame と本体の間に init_code を挿入 *)
                                  in ex_frame ^ init_code ^ code
```

### 5.2.5 アセンブリコード

```
 pushq $100
movq %rbp, %rax
leaq -8(%rax), %rax
```

```
popq (%rax)
movq %rbp, %rax
leaq -8(%rax), %rax
movq (%rax), %rax
pushq %rax
pushq $50
popq %rax
addq %rax, (%rsp)
movq %rbp, %rax
leaq -16(%rax), %rax
popq (%rax)
```

### 5.3 実行結果



# 6 課題 6

## 6.1 修正箇所の説明

累乗演算子 ^ を追加するため, 字句解析, 構文解析, 意味解析, コード生成の 4 つのファイルを修正した. アセンブリ（x86-64）には累乗を直接計算する命令が存在しないため, コード生成部で動的にループ処理を構築して計算を実現している.

- **字句解析器（lexer.mll）の修正**
  演算子の解析規則に ^ を検知する規則を追加し, 新たに定義した HAT トークンを返すようにした.
- **構文解析器（parser.mly）の修正**
  1. %token の定義に HAT を追加した.
  2. 累乗演算は掛け算や割り算よりも優先順位が高く, また一般的に右結合 (例:2^3^2 は 2^(3^2) と解釈される) であるため,TIMES や DIV の定義の下に %right HAT として優先順位と結合規則を追加した.
  3. 数式の生成規則（expr:）に | expr HAT expr を追加し, CallFunc ("^", [$1; $3]）として抽象構文木（AST）を生成するようにした.
- **意味解析（semant.ml）の修正**
  type_exp 関数内に CallFunc ("^", [left; right]）のパターンを追加し, 左辺（底）と右辺（指数）がともに整数型（INT）であるかを check_int を用いて検査する処理を記述した.
- **コード生成（emitter.ml）の修正**
  trans_exp 関数内に CallFunc ("^", [left; right]）の処理を追加した. ループ開始用（l_loop）と終了用（l_end）のラベルを incLabel() で生成し, 以下の手順でアセンブリコードを出力するようにした.
  1. 左辺（底）と右辺（指数）を評価し, それぞれスタックから %rbx と %rcx にポップする.
  2. 結果を格納する %rax を 1 で初期化する.
  3. %rcx（指数）が 0 以下になったらループを抜けて終了ラベルへジャンプする.
  4. %rax に %rbx（底）を掛け合わせる（imulq）.
  5. %rcx を 1 減らし（subq）, ループの先頭へジャンプする.

  この手順により, 指数回の反復掛け算による累乗計算を実現した.

## 6.2 ソースコード

### 6.2.1 lexer.mll の修正箇所

```
(* 修正箇所：lexer.mll の演算子定義部分 *)
| '*'                    { TIMES }
| '/'                    { DIV }
| '%'                    { MOD }
| '^'                    { HAT }  (* 追加: 累乗演算子 *)
```

### 6.2.2 parser.mly の修正箇所

```
(* 修正箇所^^e2^^91^^a0：parser.mly の %token 宣言部分 *)
%token PLUS MINUS TIMES DIV MOD HAT LB RB LS RS LP RP ASSIGN SEMI COMMA TYPE VOID

(* 修正箇所^^e2^^91^^a1：parser.mly の優先順位定義部分 *)
%left PLUS MINUS        /* lowest precedence */
%left TIMES DIV MOD     /* medium precedence */
%right HAT              /* 追加: 累乗は掛け算より優先度が高く、右結合 */
%nonassoc UMINUS        /* highest precedence */

(* 修正箇所^^e2^^91^^a2：parser.mly の expr のルール部分 *)
     | expr TIMES expr { CallFunc ("*", [$1; $3]) }
     | expr DIV expr { CallFunc ("/", [$1; $3]) }
     | expr MOD expr { CallFunc ("-", [$1; CallFunc ("*", [CallFunc ("/", [$1; $3]); $3])]) }
     | expr HAT expr { CallFunc ("^", [$1; $3]) } /* 追加: 累乗の AST 生成 */
```

### 6.2.3 semant.ml の修正箇所

```
(* 修正箇所：semant.ml の type_exp 関数の中 *)
          | CallFunc ("/", [left; right]) ->
               (check_int (type_exp left env); check_int(type_exp right env); INT)
          | CallFunc ("^", [left; right]) ->  (* 追加: 累乗の型チェック *)
               (check_int (type_exp left env); check_int(type_exp right env); INT)
```

### 6.2.4 emitter.ml

```
(* 修正箇所：emitter.ml の trans_exp 関数の中 *)
                  | CallFunc ("/", [left; right]) ->
                                          trans_exp left nest env
                                        ^ trans_exp right nest env
                                        ^ "\tpopq %rbx\n"
                                        ^ "\tpopq %rax\n"
                                        ^ "\tcqto\n"
                                        ^ "\tidivq %rbx\n"
                                        ^ "\tpushq %rax\n"

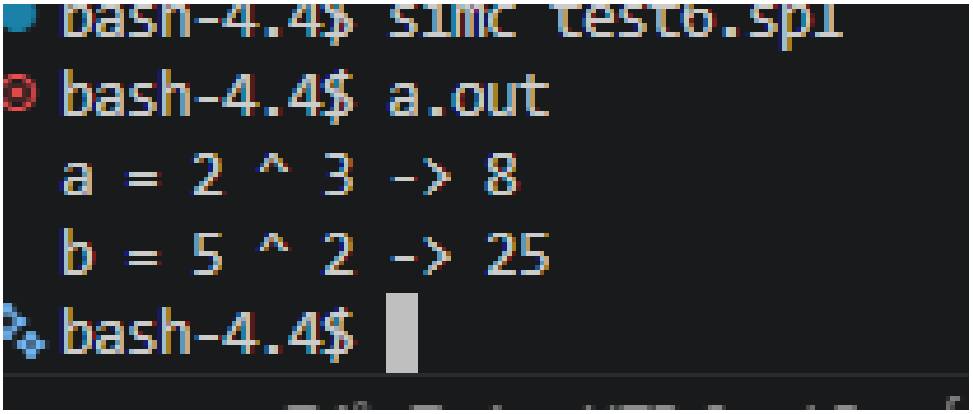
                  (* 追加: ^ のコード（ループを用いた累乗計算）*)
                  | CallFunc ("^", [left; right]) ->
                      let l_loop = incLabel() in
                      let l_end = incLabel() in
```

```
  trans_exp left nest env
^ trans_exp right nest env
^ "\tpopq %rcx\n"
^ "\tpopq %rbx\n"
^ "\tmovq $1, %rax\n"
^ sprintf "L%d:\n" l_loop
^ "\tcmpq $0, %rcx\n"
^ sprintf "\tjle L%d\n" l_end
^ "\timulq %rbx, %rax\n"
^ "\tsubq $1, %rcx\n"
^ sprintf "\tjmp L%d\n" l_loop
^ sprintf "L%d:\n" l_end
^ "\tpushq %rax\n"
```

#### 6.2.5 アセンブリコード

```
 pushq $2
pushq $3
popq %rcx
popq %rbx
movq $1, %rax
L1:
cmpq $0, %rcx
jle L2
imulq %rbx, %rax
subq $1, %rcx
jmp L1
L2:
pushq %rax
```

### 6.3 実行結果



# 7　課題 7

## 7.1　修正箇所の説明

後置インクリメント演算子 ++ を追加するため, 字句解析, 構文解析, 意味解析, コード生成の 4 つのファイルを修正した. また, 抽象構文木（AST）を新たに追加するのではなく, 既存の関数呼び出しの構造 CallFunc を利用して構文木を構築し, コード生成時に特殊なアセンブリを出力するアプローチを採用した.

- **字句解析器（lexer.mll）の修正**
  演算子の解析規則に ++ を検知する規則を追加し, 新たに定義した INC トークンを返すようにした.
- **構文解析器（parser.mly）の修正**
  1. %token の定義に INC を追加した.
  2. 後置インクリメントは単項演算子と同等の最も高い優先順位を持つため, UMINUS と同じ行に %nonassoc INC として定義した.
  3. 数式の生成規則（expr:）に | expr INC を追加し, CallFunc ("++", [$1]) として抽象構文木（AST）を生成するようにした.
- **意味解析（semant.ml）の修正**
  type_exp 関数内に CallFunc ("++", [arg]) のパターンを追加した. インクリメントの対象は値ではなく変数である必要があるため, 引数 arg が VarExp（変数）であることを確認し, さらにその変数が整数型（INT）であることを check_int で検査する処理を記述した. なお,"++" を通常の関数名として解釈させないよう, 汎用的な関数呼び出し規則である CallFunc (s, el) よりも上位にパターンマッチを記述した.
- **コード生成（emitter.ml）の修正**
  trans_exp 関数内に CallFunc("++", [VarExp v]) の処理を追加した（こちらも汎用的な関数呼び出し規則の直上に配置した）. 以下の手順でアセンブリコードを出力するようにした.
  1. trans_var を用いて変数 v のアドレスを計算し,%rax に格納する.
  2. %rax のアドレスから現在の値（増加前の値）を %rbx に読み込む.
  3. この式自体の評価結果は「増加前の値」となるため,%rbx の値をスタックにプッシュする.
  4. %rbx の値を 1 加算（addq $1, %rbx）する.
  5. 加算後の %rbx の値を, 再び %rax のアドレス（メモリ上の変数領域）へ書き戻す.

## 7.2　ソースコード

### 7.2.1　lexer.mll の修正箇所

```
(* 修正箇所：lexer.mll の演算子定義部分 *)
| '^'                     { HAT }
| "++"                    { INC }  (* 追加: インクリメント演算子 *)
```

### 7.2.2 parser.mly の修正箇所

```
(* 修正箇所^^e2^^91^^a0：parser.mly の %token 宣言部分 *)
%token PLUS MINUS TIMES DIV MOD HAT INC LB RB LS RS LP RP ASSIGN SEMI COMMA TYPE VOID

(* 修正箇所^^e2^^91^^a1：parser.mly の優先順位定義部分 *)
%right HAT              /* medium-high precedence */
%nonassoc UMINUS INC    /* 修正: 単項マイナスと同じ最も高い優先順位に INC を追加 */

(* 修正箇所^^e2^^91^^a2：parser.mly の expr のルール部分 *)
     | expr HAT expr { CallFunc ("^", [$1; $3]) }
     | expr INC { CallFunc ("++", [$1]) } /* 追加: 後置インクリメントの AST 生成 */
```

### 7.2.3 semant.ml の修正箇所

```
(* 修正箇所：semant.ml の type_exp 関数の中 *)
          | CallFunc ("!", [arg]) ->
               (check_int (type_exp arg env); INT)

          (* 追加: ++ の型チェック（必ず CallFunc (s, el) より上に配置） *)
          | CallFunc ("++", [arg]) ->
               (match arg with
                | VarExp v -> (check_int (type_var v env); INT)
                | _ -> raise (TypeErr "lvalue required for ++"))

          (* （参考）既存の関数呼び出しルール *)
          | CallFunc (s, el) ->
               let entry = env s in
```

### 7.2.4 emitter.ml

```
(* 修正箇所：emitter.ml の trans_exp 関数の中 *)
                 | CallFunc("!",  arg::_) ->
                                          trans_exp arg nest env
                                        ^ "\tnegq (%rsp)\n"

                 (* 追加: ++ のコード（必ず CallFunc (s, el) より上に配置） *)
                 | CallFunc("++", [VarExp v]) ->
                                          trans_var v nest env
                                        ^ "\tmovq (%rax), %rbx\n"
```

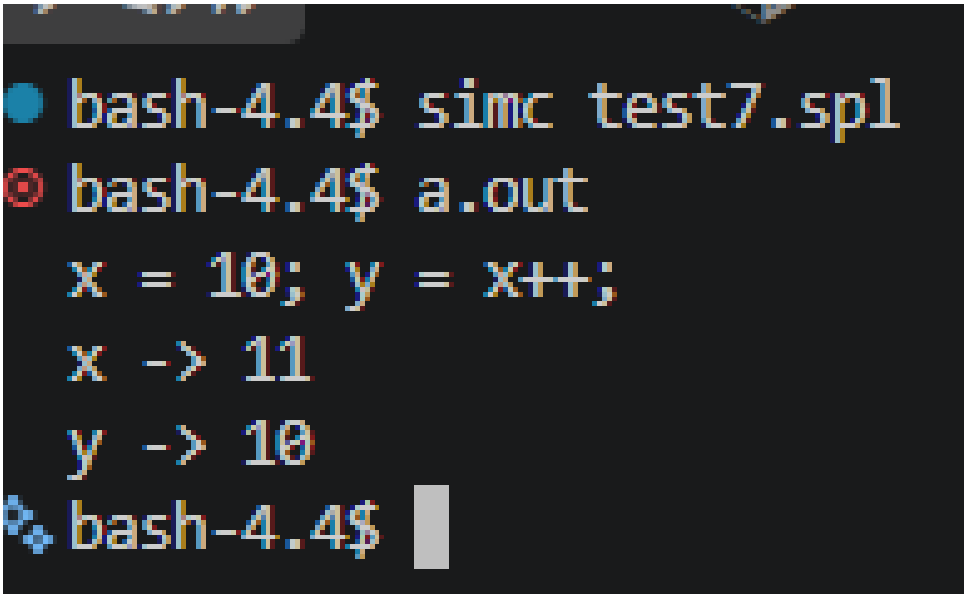
```
                                    ^ "\tpushq %rbx\n"
                                    ^ "\taddq $1, %rbx\n"
                                    ^ "\tmovq %rbx, (%rax)\n"

    (* （参考）既存の関数呼び出しルール *)
    | CallFunc (s, el) ->
                    trans_stmt (CallProc(s, el)) nest initTable env
```

### 7.2.5 アセンブリコード

```
 movq %rbp, %rax
leaq -8(%rax), %rax
movq (%rax), %rbx
pushq %rbx
addq $1, %rbx
movq %rbx, (%rax)
```

## 7.3 実行結果



# 8 課題 8

## 8.1 修正箇所の説明

複合代入演算子 += を追加するにあたり, 抽象構文木（AST）を新たに拡張するのではなく, 構文解析の段階で **変数 += 式;** を **変数 = 変数 + 式;** という既存の構文（代入文と加算式）に展開する構文糖衣（シンタックスシュガー）として実装した. この実装のため, lexer.mll と parser.mly の 2 ファイルを修正した.

- **字句解析器（lexer.mll）の修正**
  演算子の解析規則に += を検知する規則を追加し, 新たに定義した PLUSEQ トークンを返すようにした.
- **構文解析器（parser.mly）の修正**
  1. %token の定義に PLUSEQ を追加し, 新しいトークンとして宣言した（文の構成要素として使うため, 優先順位の定義は不要である）.
  2. 文の生成規則（stmt:）に, 通常の変数に対する ID PLUSEQ expr SEMI の規則を追加し, そのアクションとして Assign (Var $1, CallFunc ("+", [VarExp (Var $1); $3])) を返すように記述した. これにより,a += 5; が a = a + 5; の AST に自動変換される.
  3. 配列などの添字付き変数（ID LS expr RS）に対する += も同様にサポートするため,ID LS expr RS PLUSEQ expr SEMI の規則を追加し, 同様に加算と代入を組み合わせた AST を生成するようにした.

## 8.2 ソースコード

### 8.2.1 lexe.mll の修正箇所

```
(* 修正箇所：lexer.mll の演算子定義部分 *)
| "++"                    { INC }
| "+="                    { PLUSEQ }  (* 追加: 複合代入演算子 *)
```

### 8.2.2 parser.mly の修正箇所

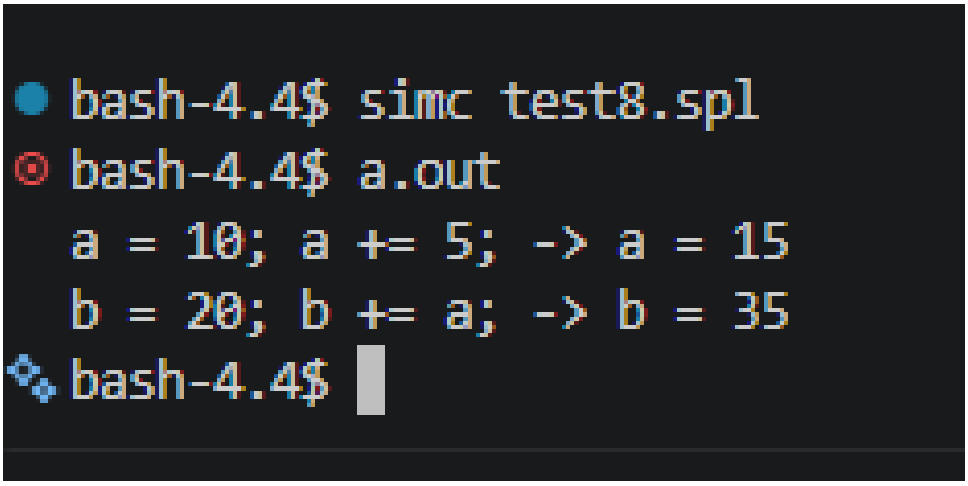
```
(* 修正箇所^^e2^^91^^a0：parser.mly の %token 宣言部分 *)
%token PLUS MINUS TIMES DIV MOD HAT INC PLUSEQ LB RB LS RS LP RP ASSIGN SEMI COMMA TYPE VOID

(* 修正箇所^^e2^^91^^a1：parser.mly の stmt のルール部分 *)
stmt : ID ASSIGN expr SEMI    { Assign (Var $1, $3) }
     | ID PLUSEQ expr SEMI    { Assign (Var $1, CallFunc ("+", [VarExp (Var $1); $3])) } /* 追
加: 変数 += 式; を 変数 = 変数 + 式; に変換 */
     | ID LS expr RS ASSIGN expr SEMI  { Assign (IndexedVar (Var $1, $3), $6) }
     | ID LS expr RS PLUSEQ expr SEMI  { Assign (IndexedVar (Var $1, $3), CallFunc ("+", [VarExp (In
加: 配列 += 式; の対応 */
```

### 8.2.3 アセンブリコード

```
 movq %rbp, %rax
leaq -8(%rax), %rax
movq (%rax), %rax
pushq %rax
pushq $5
popq %rax
addq %rax, (%rsp)
movq %rbp, %rax
leaq -8(%rax), %rax
popq (%rax)
```

## 8.3 実行結果



# 9 課題９

## 9.1 修正箇所の説明

「条件判定を行う前に, 必ず 1 回は文を実行する」という do-while 文のセマンティクスを実現するため, 抽象構文木（AST）に新たなノードを追加し, コンパイラの各フェーズを拡張した.

- **抽象構文木（ast.ml）の修正**
  文を表すデータ型 stmt に, 新たに DoWhile of stmt * exp（実行する文と, ループの継続条件式）を追加した.
- **字句解析器（lexer.mll）の修正**
  do キーワードを検知し, 新たなトークン DO を返す規則を追加した.
- **構文解析器（parser.mly）の修正**
  1. %token 宣言に DO を追加した.
  2. 文の生成規則（stmt:）に DO stmt WHILE LP cond RP SEMI の並びを検知する規則を追加し,DoWhile（$2, $5）の抽象構文木を生成するように記述した.
- **意味解析（semant.ml）の修正**
  type_stmt 関数において,DoWhile（_, e）のパターンを追加し, 既存の While 文と同様に type_cond e env を呼び出して条件式 e の型チェックを行う処理を記述した.
- **コード生成（emitter.ml）の修正**
  trans_stmt 関数に DoWhile（s, e）のアセンブリ生成処理を追加した.
  1. ループの先頭を表すラベル（l_start）を出力する.
  2. まずブロック内の文 s を実行するコード（trans_stmt s）を出力する.
  3. 条件式 e を評価するコード（condCode）を出力する. この condCode には, 条件が False だった場合にループ終了ラベル（l_end）へジャンプする命令が含まれている.
  4. 条件が True の場合（ジャンプしなかった場合）は,jmp L_start でループの先頭に戻る.
  5. ループ終了ラベル（l_end）を出力する.

  この構造により,「最低 1 回は実行され, 条件を満たす限り繰り返す」アセンブリコードを出力できるようになった.

## 9.2 ソースコード

### 9.2.1 ast.ml の修正箇所

```
(* 修正箇所：ast.ml の stmt の定義部分 *)
and stmt = Assign of var * exp
         | CallProc of id * (exp list)
         | Block of (dec list) * (stmt list)
         | If of exp * stmt * (stmt option)
         | While of exp * stmt
         | DoWhile of stmt * exp   (* 追加: do-while 文 *)
```

```
        | NilStmt
```

### 9.2.2 lexer.mll の修正箇所

```
(* 修正箇所：lexer.mll のキーワード定義部分 *)
| "while"                 { WHILE }
| "do"                    { DO }    (* 追加: do キーワード *)
```

### 9.2.3 parser.mly の修正箇所

```
(* 修正箇所^^e2^^91^^a0：parser.mly の %token 宣言部分 *)
%token INT IF WHILE DO SPRINT IPRINT SCAN EQ NEQ GT LT GE LE ELSE RETURN NEW
(* DO を追加 *)

(* 修正箇所^^e2^^91^^a1：parser.mly の stmt のルール部分 *)
     | IF LP cond RP stmt ELSE stmt
                              { If ($3, $5, Some $7) }
     | WHILE LP cond RP stmt  { While ($3, $5) }
     | DO stmt WHILE LP cond RP SEMI { DoWhile ($2, $5) } /* 追加: do-while 構文 */
     | SPRINT LP STR RP SEMI  { CallProc ("sprint", [StrExp $3]) }
```

### 9.2.4 semant.ml の修正箇所

```
(* 修正箇所：semant.ml の type_stmt 関数の中 *)
          | Assign (v, e) ->
               if (type_var v env) != (type_exp e env) then raise (TypeErr "type error 4")
          | If (e,_,_) -> type_cond e env
          | While (e,_) -> type_cond e env
          | DoWhile (_,e) -> type_cond e env  (* 追加: 条件式の型チェック *)
          | NilStmt -> ()
```

### 9.2.5 emitter.ml の修正箇所

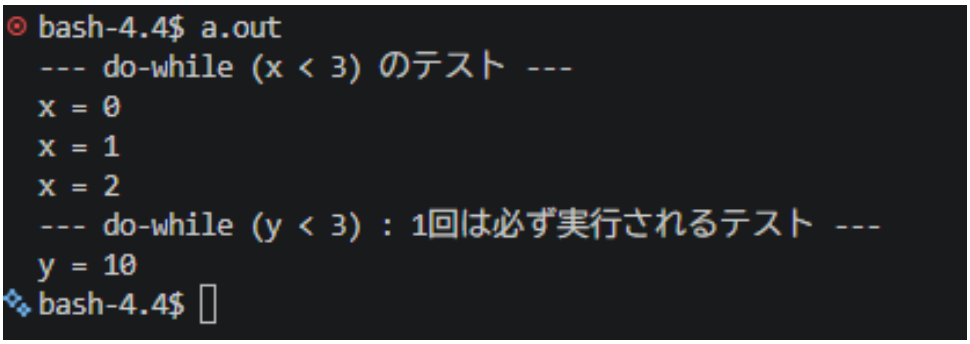
```
(* 修正箇所：emitter.ml の trans_stmt 関数の中 *)
                  | While (e,s) -> let (condCode, l1) = trans_cond e nest env in
                                   let l2 = incLabel() in
                                       sprintf "L%d:\n" l2
                                     ^ condCode
                                     ^ trans_stmt s nest tenv env
                                     ^ sprintf "\tjmp L%d\n" l2
                                     ^ sprintf "L%d:\n" l1
```

```
(* 追加: do-while 文のコード *)
| DoWhile (s, e) ->
    let l_start = incLabel() in
    let (condCode, l_end) = trans_cond e nest env in
        sprintf "L%d:\n" l_start
      ^ trans_stmt s nest tenv env
      ^ condCode
      ^ sprintf "\tjmp L%d\n" l_start
      ^ sprintf "L%d:\n" l_end
```

#### 9.2.6 アセンブリコード

```
 L2:
movq %rbp, %rax
leaq -8(%rax), %rax
movq (%rax), %rax
pushq %rax
pushq $1
popq %rax
addq %rax, (%rsp)
movq %rbp, %rax
leaq -8(%rax), %rax
popq (%rax)
movq %rbp, %rax
leaq -8(%rax), %rax
movq (%rax), %rax
pushq %rax
pushq $3
popq %rax
popq %rbx
cmpq %rax, %rbx
jge L3
jmp L2
L3:
```

### 9.3 実行結果


bash-4.4$ a.out
--- do-while (x < 3) のテスト ---
x = 0
x = 1
x = 2
--- do-while (y < 3) : 1回は必ず実行されるテスト ---
y = 10
bash-4.4$

# 10　課題 10

## 10.1　修正箇所の説明

**for（変数 = 式 1 .. 式 2）文** というループ構文を追加するため, 字句解析器と構文解析器を修正した. この構文は「1, 変数を式 1 で初期化し,2, 変数が式 2 と等しくない間, 文を実行し,3, 変数を +1 する」という処理手順に分解できる. そのため, 抽象構文木（AST）やコード生成部に新たな構造を追加するのではなく, 構文解析の段階で既存の「代入文」と「while 文」の組み合わせに自動変換する構文糖衣（シンタックスシュガー）として実装するアプローチを採用した.

- **字句解析器（lexer.mll）の修正**
  キーワード for と, 範囲指定に用いる .. を検知する規則を追加し, それぞれ新しいトークン FOR および DOTDOT を返すようにした.
- **構文解析器（parser.mly）の修正**
  1. %token の定義に FOR と DOTDOT を追加し, 新しいトークンとして宣言した.
  2. 文の生成規則（stmt:）に FOR LP ID ASSIGN expr DOTDOT expr RP stmt のパターンを追加した.
  3. このアクション部において, 生成される抽象構文木が Block となり, その中に「式 1 の代入（初期化）」と「式 2 と等しくないかを判定する While 文」が生成されるように記述した. さらに While 文の本体の最後には, ループ変数を +1 する代入文（Assign）が自動的に挿入されるようにした.

この工夫により, 意味解析（semant.ml）やコード生成（emitter.ml）に一切手を加えることなく, 新しいループ構文を実装することができた.

## 10.2　ソースコード

### 10.2.1　lexer.mll の修正箇所

```
(* 修正箇所：lexer.mll のキーワード・記号定義部分 *)
| "do"                    { DO }
| "for"                   { FOR }       (* 追加: for キーワード *)
| ".."                    { DOTDOT }    (* 追加: 範囲指定の記号 *)
```

### 10.2.2　parser.mly の修正箇所

```
(* 修正箇所^^e2^^91^^a0：parser.mly の %token 宣言部分 *)
%token INT IF WHILE DO FOR SPRINT IPRINT SCAN EQ NEQ GT LT GE LE ELSE RETURN NEW
%token PLUS MINUS TIMES DIV MOD HAT INC PLUSEQ DOTDOT LB RB LS RS LP RP ASSIGN SEMI COMMA TYPE VOID

(* 修正箇所^^e2^^91^^a1：parser.mly の stmt のルール部分 *)
     | DO stmt WHILE LP cond RP SEMI { DoWhile ($2, $5) }
```

```
/* 追加: for 文を「初期化の代入」と「while ループ」の組み合わせの AST に展開する */
| FOR LP ID ASSIGN expr DOTDOT expr RP stmt
    {
      Block ([], [
        Assign (Var $3, $5);
        While (
          CallFunc ("!=", [VarExp (Var $3); $7]),
          Block ([], [
            $9;
            Assign (Var $3, CallFunc ("+", [VarExp (Var $3); IntExp 1]))
          ])
        )
      ])
    }
```

#### 10.2.3 アセンブリコード

```
 pushq $0
movq %rbp, %rax
leaq -8(%rax), %rax
popq (%rax)
L3:
movq %rbp, %rax
leaq -8(%rax), %rax
movq (%rax), %rax
pushq %rax
pushq $3
popq %rax
popq %rbx
cmpq %rax, %rbx
je L2

movq %rbp, %rax
leaq -8(%rax), %rax
movq (%rax), %rax
pushq %rax
pushq $1
popq %rax
addq %rax, (%rsp)
movq %rbp, %rax
```

```
leaq -8(%rax), %rax
popq (%rax)
jmp L3
L2:
```

### 10.3 実行結果

